Travaux Pratiques Data Mining & Deep Learning

Classification de Texte et Analyse de Sentiments :
Naive Bayes, RNN, LSTM et Transformers

Boudaoud Sid Ahmed

24 octobre 2026

---

1 TP4 : Analyse de Sentiments sur Sentiment140 De Naive Bayes aux Transformers

Objectifs

- Comprendre le principe du classifieur probabiliste Naive Bayes appliqué au texte.
- Prétraiter et nettoyer un corpus volumineux issu des réseaux sociaux (**Sentiment140**).
- Implémenter et entraîner des architectures séquentielles récurrentes (**RNN Simple** et **LSTM**).
- Mettre en uvre un modèle basé sur le mécanisme d'attention (**Transformer** / DistilBERT).
- Comparer expérimentalement les performances des quatre approches et justifier scientifiquement les résultats obtenus.

Contexte

Dans ce TP, vous allez travailler sur le célèbre dataset **Sentiment140**, qui regroupe 1,6 million de tweets extraits via l'API Twitter et annotés selon leur polarité : `0` pour un sentiment *négatif* et `4` (à recoder en `1`) pour un sentiment *positif*. Pour des raisons de temps de calcul, vous pourrez extraire un sous-échantillon aléatoire parfaitement équilibré (par exemple entre 50 000 et 100 000 tweets).

Contrairement aux messages classiques, les tweets contiennent des abréviations, des mentions (`@user`), des liens URL, des hashtags, ainsi que des structures syntaxiques complexes (négations, ironie, sarcasme). L'objectif est de construire, évaluer et confronter quatre familles de modèles afin d'analyser comment chaque architecture traite la séquentialité et le contexte sémantique.

Vous devez effectuer :

- la préparation et le nettoyage spécifique des tweets,
- l'extraction des caractéristiques (*Bag-of-Words*, *TF-IDF*, *Word Embeddings*, *Subword Tokenization*),
- l'entraînement des quatre modèles : **Naive Bayes**, **RNN Simple**, **LSTM** et **Transformer**,
- l'évaluation comparative et la justification théorique et empirique des performances.

Consignes

Partie 1 : Préparation des données et Classifieur Naive Bayes (Baseline)

1. Charger le dataset **Sentiment140**, explorer sa structure et vérifier la distribution des classes.
2. Nettoyer les tweets : suppression des mentions (`@username`), des URLs, des caractères spéciaux et normalisation en minuscules. *Attention : étudier l'impact de la conservation ou de la suppression des stopwords de négation (`not`, `no`, `never`).*
3. Transformer les messages en vecteurs numériques via *Bag-of-Words* (`CountVectorizer`) puis *TF-IDF* (`TfidfVectorizer` avec unigrammes et bigrammes).
4. Entraîner un classifieur Naive Bayes (`MultinomialNB`).
5. Identifier les 15 mots (ou bigrammes) les plus discriminants pour chaque polarité.

Partie 2 : Réseaux de Neurones Récurrents (RNN Simple et LSTM)

6. Tokeniser le corpus sous forme de séquences d'entiers et appliquer un remplissage (*padding*) pour obtenir des séquences de longueur fixe (ex. `max_len = 40`).
7. Construire et entraîner un **RNN Simple** (`SimpleRNN`) précédé d'une couche d'`Embedding`. Observer les courbes d'apprentissage (perte et accuracy sur les jeux d'entraînement et de validation).
8. Construire et entraîner un réseau **LSTM** (ou `Bi-LSTM`) en conservant le même découpage d'évaluation afin de garantir une comparaison équitable.

Partie 3 : Architecture à Mécanisme d'Attention (Transformers)

9. Utiliser le tokeniseur sous-lexical (*WordPiece*) associé à un modèle Transformer pré-entraîné léger (ex. `distilbert-base-uncased`).
10. Effectuer un apprentissage par transfert (*fine-tuning*) du modèle Transformer sur le jeu d'entraînement pendant 2 à 3 époques.
11. Tester qualitativement les 4 modèles sur un jeu de 5 phrases complexes rédigées par vos soins (contenant une double négation, une conjonction d'opposition *ń mais ż* ou du sarcasme).

Partie 4 : Évaluation Comparative et Synthèse

12. Évaluer chacun des quatre modèles sur le même jeu de test à l'aide des métriques suivantes : *Accuracy*, *Précision*, *Rappel*, *F1-score*, *Matrice de confusion* et *Temps d'entraînement*.
13. Remplir le tableau récapitulatif ci-dessous dans votre compte-rendu :

Modèle	**Représentation**	**Accuracy**	**Précision**	**Rappel**	**F1-Score**	**Temps**
Naive Bayes	BoW / TF-IDF	...	...	...	...	...
RNN Simple	Séquence + Embedding	...	...	...	...	...
LSTM / Bi-LSTM	Séquence + Embedding	...	...	...	...	...
Transformer	Subword + Attention	...	...	...	...	...

Questions d'Analyse et de Justification

1. **Hypothèse de Naive Bayes :** Pourquoi dit-on que Naive Bayes repose sur une hypothèse ń naïve ż ? En quoi cette hypothèse et la représentation en sac de mots limitent-elles la détection des inversions de polarité (ex. *ń not good at all ż*) ?
2. **RNN Simple vs LSTM :** Comment expliquez-vous la différence de performance entre le RNN Simple et le LSTM, en particulier sur les tweets longs ? Justifiez votre réponse en détaillant le problème de disparition du gradient (*vanishing gradient*) et le rôle des portes (*forget gate*, *input gate*, *output gate*) de la cellule LSTM.
3. **Apport des Transformers :** Pourquoi l'architecture Transformer surpasse-t-elle les modèles récurrents sur Sentiment140 ? Appuyez votre justification sur le mécanisme de *Self-Attention*, le contexte bidirectionnel, la tokenisation sous-lexicale face aux mots hors-vocabulaire (argot, fautes de frappe) et le pré-entraînement.
4. **Analyse des erreurs :** Quels types d'erreurs vos modèles commettent-ils le plus ? Le passage au LSTM puis au Transformer permet-il de corriger les faux positifs et faux négatifs liés aux négations et à l'ironie ? Illustrez avec des exemples concrets issus du jeu de test.
5. **Compromis Performance / Coût de calcul :** En comparant le gain en F1-score par rapport au temps d'entraînement et aux ressources matérielles requises (CPU vs GPU), dans quelles situations réelles privilégieriez-vous malgré tout un modèle Naive Bayes ou LSTM plutôt qu'un Transformer ?